

Biology 3315 Molecular Evolution and
Phylogenetics
Lab manual

Dr. Ido Hatam

February 13, 2023

Contents

An introduction to the lab	5
How to get the most out of the lab	5
Lab 1 introduction to the R programming language and the R studio IDE	7
Learning objectives	7
Getting started	7
Programming in R - the basics	14
Data types (objects) in R	20
Summary exercise Lab1	44
1 Cross-references	47
1.1 Chapters and sub-chapters	47
1.2 Captioned figures and tables	47
2 Parts	51
3 Footnotes and citations	53
3.1 Footnotes	53
3.2 Citations	53
4 Blocks	55
4.1 Equations	55
4.2 Theorems and proofs	55
4.3 Callout blocks	55
5 Sharing your book	57
5.1 Publishing	57
5.2 404 pages	57
5.3 Metadata for sharing	57

An introduction to the lab

This is a *sample* book written in **Markdown**. You can use anything that Pandoc's Markdown supports; for example, a math equation $a^2 + b^2 = c^2$.

How to get the most out of the lab

Each **bookdown** chapter is an .Rmd file, and each .Rmd file can contain one (and only one) chapter. A chapter *must* start with a first-level heading: **# A good chapter**, and can contain one (and only one) first-level heading.

Use second-level and higher headings within chapters like: **## A short section** or **### An even shorter section**.

The **index.Rmd** file is required, and is also your first book chapter. It will be the homepage when you render the book.

Lab 1 introduction to the R programming language and the R studio IDE

Learning objectives

Click to expand

By the end of this lab you will be able to:

- Install the R interpreter and the Rstudio integrated development environment
- Demonstrate the ability to install and load packages from a variety of repositories including development versions from github
- Understand the different components of the Rstudio environment
- Demonstrate mastery of R object and data types including indexing, subsetting and manipulating objects
- Demonstrate the ability to call and chain functions

Getting started

What is R?

R is a free and open source programming language, operating as a GUN project. The R language is maintained and supported by the R Foundation for Statistical Computing.

“R is a functional programming language and environment for statistical computing and graphics”

— The R dev team

R is best known as a programming language with strong capabilities of manipulation, exploration and statistical analysis of data from diverse types and sources. However, it also provides an environment for data handling, graphical representation of data as well as data reporting. R is a great platform for documentation and in fact this entire lab manual was written in R using the markdown package.

Why use R?

- Open source and free
 - Available on all major platform
- Large community of users and developers (over 2M users in academia, government, and industry)
- Thousands of open source packages aimed at the analysis of biological data in general and genomic and evolutionary data in particular
- Customizable and allows for the construction of pipelines
- Reproducible analysis

Downloading and installing R and RStudio desktop

To download R, go to the R download page select the SFU mirror then select your operating system and download and install both 32 and 64 bit versions.

To download RStudio desktop, go to the RStudio download page and download the free RStudio desktop version.

Note that you must download and install both R and RStudio in order to be able to work with R in the RStudio IDE. Note that it is important that you have the most up to date version of R, even if R is already installed on your computer make sure to update it. If using Windows you can use the `installr` package to update R.

The R interpreter

Generally speaking, R is an interpreted language, and it's human readable code cannot be directly converted to machine code by the operating system. Therefore, it needs an interpreter to translate the human readable code to a format understood by the operating system and the processing unit as the code runs.

The R console is an R interpreter with a very simple GUI. It allows you to write and run your code line by line just like you would in a terminal.

Console input and output:

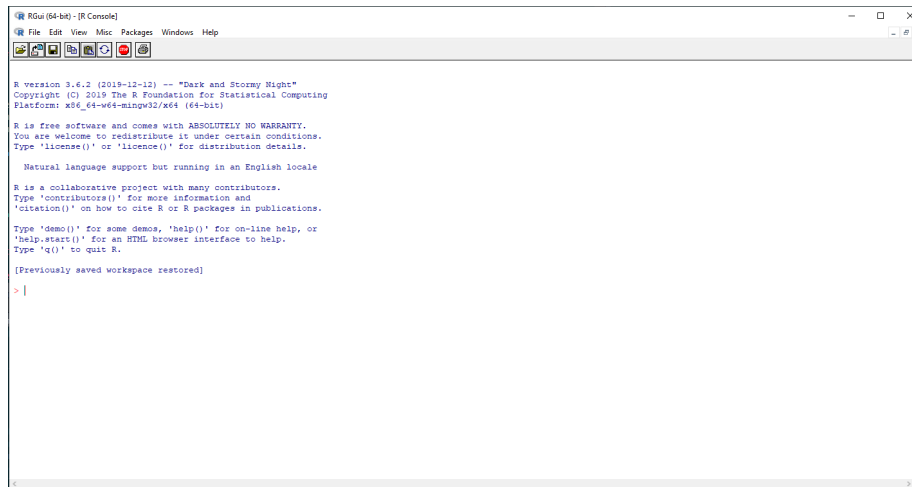


Figure 1: The R console

```
> 1+1
```

```
[1] 2
```

```
> 3*3
```

```
[1] 9
```

```
> 4/2
```

```
[1] 2
```

```
> 4^2
```

```
[1] 16
```

```
> print("Hello world")
```

```
[1] "Hello world"
```

In the example code above... something about the console interpreting white and in gray output... Furthermore, the print function was called to print the famous “Hello world” combo no coding workshop is complete without.

Short console exercise

Now let’s get you to work. The following lines of code will assign values to the variables “j”, “k”, and “rowling” (we will discuss variable assignment in R later in this lab). Open the R console on your computer and run them.

```
> j <- "Harry"
> k <- "Poter"
> rowling <- data.frame(First_name = j, Last_name = k)
```

Notice that no output was generated, since you only assigned values to some variables. You can get the interpreter to output the value of each variable by calling them in the console. Try it for each of them. As you can see, the variables are stored in memory. This is evident by the output generated when you called the variables in the console. Furthermore, without knowing too much R, you probably already guessed that not all variables are created equal and that there is some differences between rowling and the rest of the variables. However, the console doesn't let one see immediately how many variables one stored in memory, nor does it let one see their names, memory allocation, type, etc. In order to see those using just the console you would have to call an array of functions. For example try running the following function in your console.

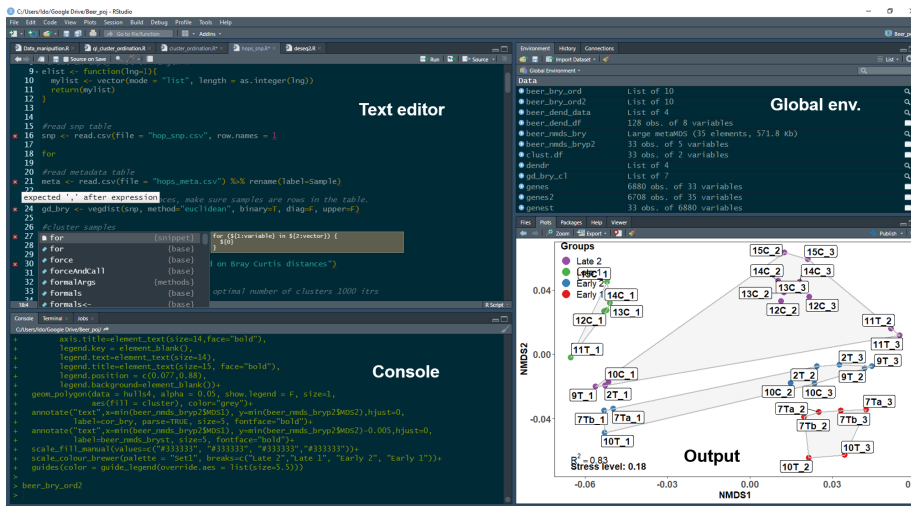
```
> ls()
```

If you are familiar with UNIX based operating systems, you probably recognized that ls is short for list. When running the ls() function in the console, the the output will list all stored variables. But as you may notice, this is about all it does. You can also imagine that this is not the most efficient way to keep track of the variables you declare when working with more than a couple variables (which is sometimes required). In order to keep things streamlined and organized, it is advisable to work with an Integrated Development Environment (IDE), which allows you to have better control of code editing, variable storage, console output etc. R has several IDEs, however the most popular by far is RStudio.

The RStudio IDE

As mentioned above, RStudio is an integrated development environment (IDE) for R. It is highly customizable but its default setting include four panes, a text editor pane, a console pane, a global environment pane, and an output pane.

Unlike the basic console GUI, RStudio allows you to manage and separate different R projects (e.g. you could have a separate project for this course and for a stats course). Another advantage of R studio is its integration of version control which include both Git and SVN. This will be beneficial if future course when you will be required to collaborate on projects.



The text editor pane

The text editor pane is found on the top left corner of the RStudio screen. The text editor allows you to edit code before sending it to the console. As shown in the figure below, there are some advantages of writing your code with the RStudio code editor. It auto-completes syntax, functions, and variables. The text editor also helps in code debugging by flagging lines with errors (marked by a red dot with an X). It also allows you to edit multiple scripts using tabs, just like your web browser.

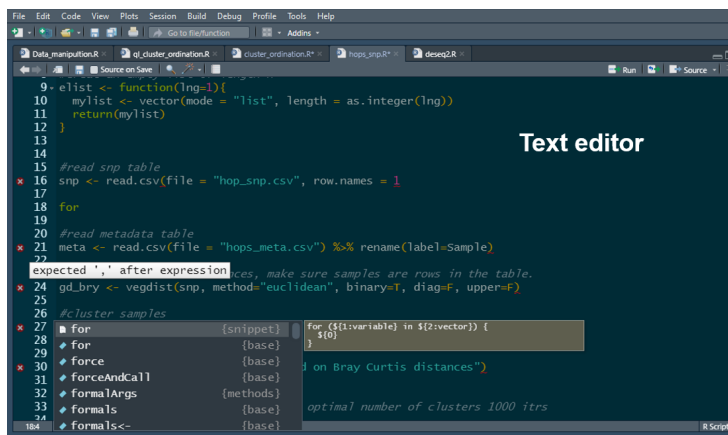


Figure 2: The RStudio text editor

Using the text editor allows you to send code to the console line by line (or as chunks, by highlighting multiple lines) by using a keyboard shortcut(command

enter in mac-OS, Ctrl enter win and Linux). Alternatively this can also be done pressing the run button at the top left corner of the editor. The editor also allows you to source the entire script to the console by pressing the source button (the rightmost button). The text editor supports multiple languages(e.g. C++, python, SQL, etc...) if you fancy programming in a language other than R.

The console pane

The console pane is found below the text editor. This is where code input and output resulting from the code can be seen. You can also write and run code directly at the console, just like when using the R interpreter GUI. The console allows you to manage multiple R jobs (we will not use that) through the job tab, as well as running a bash terminal (terminal tab) for system commands (we might use that).

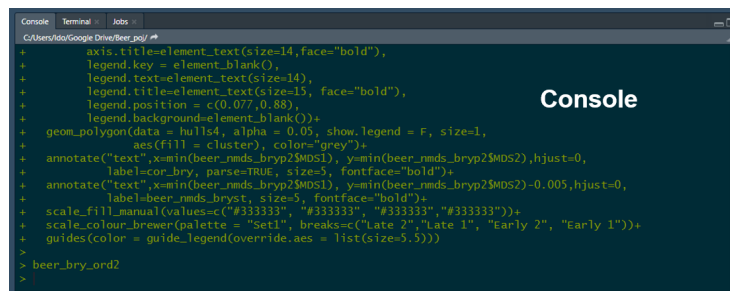
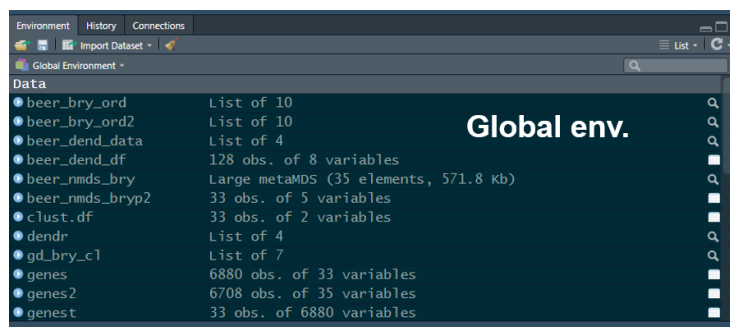


Figure 3: The RStudio console

The global environment pane

The global environment pane is found to the right of the text editor, this is where you can see all the variables you called. It will also tell you what type of variables they are and how much memory they use.

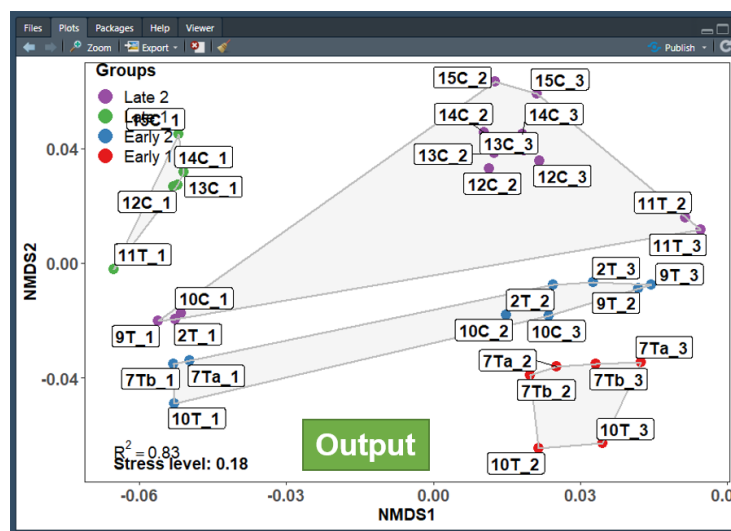


The global environment pane also contains a history and a connection tab. The history tab saves all the lines of code ran through the console. It also allows

you to resend lines of code back to the text editor which is very useful if you deleted them by accident instead of commenting them out. The connection tab allows you to connect RStudio to database servers (e.g. SQL (ODBC) or Spark, we will not use that).

The output pane

The output pane is found below the global environment pane and is where graphical output will show up. Here you can scroll through as well as export all of the plots that you generated while analyzing data and running scripts.



The output pane also includes a file tab which allows you to see all the files you exported as well as a help tab which allows you to brows through documentation with details of the functions you might want to run. The package tab helps you manage the R extension packages you might have installed and allows you to see which ones are loaded.

Note that each of these panes include tabs that were not included in this tutorial. However, they can be very useful and I encourage you to explore them as you learn to use RStudio.

Short exercise in RStudio

Copy the following code to RStudio's text editor

```
> j <- "Harry"
> k <- "Potter"
```

```

> rowling <- data.frame(First_name = j, Last_name = k)
>
> data(cars)
> car_data <- cars
> plot(cars)

> Useless.function <- function(x){
+   if(is.numeric(x)){
+     print("This function is goddamn useless")
+   } else {
+     print("This function is goddamn useless")
+   }
+ }

```

- Try writing the last line instead of copying and pasting it, did you notice how the editor completes the syntax for you as the function `data.frame` is called?
- Practice running it line by line, as a chunk, and sourcing it completely.
- What happens in the global environment when you ran the code?
- Are all called variables listed under the same category in the global environment pane?
- Can you tell how much memory is `car_data` using?
- What happens when you press `rowling`?
- What do you see in the output pane?
- What happens when you call the `rowling` variable directly in the console?

Programming in R - the basics

R language and syntax

R is an expression language with very simple syntax requiring that you list call-functions, in order to perform tasks on your data.

The code snippet below calls two functions. The first function is mathematical operator `+`, which has an irregular function structure (infix function if you're interested) that tells R to sum two numbers. In this case, the arguments (the numbers needed to be summed) are placed on both side of the function (the `+` operator). These are referred to as left hand side (LHD) and right hand side (RHD) arguments. The second function is `print()`, which is a regular function. For regular functions R syntax requires that you write the arguments within the parenthesis that come after the function.

The code snippet also includes hashed text (lines of text beginning with #), hasing tells R to ignore the line. Hasing can be used to leave comments on the text.

```
> #Calling an irregular function <-- btw this is how you comment
> 1+1
```

```
[1] 2
```

```
> #Calling a regular function
> print(j)
```

```
[1] "Harry"
```

The code snippet above is an example for a script where functions are separated by a line, and functions are called sequentially. This can also be achieved by separating the functions with semicolon (;). The example below gives identical results to the previous one however this writing style reduces the readability of the code and is therefore discouraged.

```
> #An example of poor style
> 1+1;print(j)
```

```
[1] 2
```

```
[1] "Harry"
```

R is case sensitive, whenever a function is called one needs to make sure that both it and its arguments are written properly, otherwise the function will produce an error.

```
> #Variable is capitalized
> print(J)
```

```
Error in print(J): object 'J' not found
```

```
> #Function name is capitalized
> Print(j)
```

```
Error in Print(j): could not find function "Print"
```

The `print()` function only accepts one argument, however some functions in R accept multiple arguments. The code below calls the function `c()`, which concatenates similar variables in R into one variable. When more than one argument is used, the arguments should be separated by a comma (,) unless it is an irregular function e.g. mathematical and logical operators.

```
> #Spoiler for the order of operations in R
> print(c(j,k))
```

```
[1] "Harry" "Poter"
```

Here is another example of a function that accepts more than one argument. The `sep` argument is an example of an argument that's not a variable.

```
> #j and k are variables, sep (short for separator) is not.  
>  
> print( paste(j,k, sep = ";") )
```

```
[1] "Harry;Poter"
```

Seeking help

So how can you tell which or how many arguments does a function accept? Well, the RStudio text editor offers a glimpse of the arguments as you write the function but the proper way to do is to call the function documentation. This is done by invoking the help function, which uses the ? operator as seen in the code snippet below.

```
> ?paste  
>  
> # This is equivalent of help(paste)  
> #? is another irregular function that accepts a LHS argument
```

When running this code using RStudio, the help tab will be prompted and you will be able to see the paste function documentation. Arguments for the function are specified both under usage and arguments. Usage specifies the arguments and informs the user of arguments with set values, for example the argument sep has a set value of " ", which denotes white space. Often you would encounter arguments with a set value of NULL, this lets R know that for all intents and purposes the argument should be ignored unless value is specified by the user. The arguments section provides an explanation regarding to the function of each of the arguments. For example the value sep is a character string that separates the terms being pasted. This means that the default separator for pasted terms is a white space.

The ... (dot dot dot) argument

The paste function also includes the "..." argument, this is a special argument that matches any unspecified arguments and makes sure that the function accepts them if possible. Here, the underlying condition is that any unspecified argument must be a vector (we will discuss what vectors are later in this tutorial).

Extended help search

Sometimes the documentation for a function is not found in the environment you loaded (we will get to libraries of functions extending the capabilities of R later in this tutorial), in order to perform an extended search of all libraries installed on your computer use double question mark (??).

Quick R exercise - basic syntax:

1. The previous code snippets provided a good example of the order of operation in R when nesting functions. Look at the code below, what do you think the order of operation is? Which function will run first? Which will run second, which will run third?

```
> print(paste(c(j,k),  
+           c(k,j), sep = " "))
```

2. sum is a function in R. Can you find its purpose and how many arguments does it accept?

A Note on indentation in R

The example code above includes indentation however, unlike in other programming languages, R does not require that you indent your code. Indentation is only used to make code more readable to humans, especially as it gets more complex. As you will learn, the flow of operations in the basic R is determined by parenthesis “()” when combining functions (e.g. the code above) and braces “{}” when writing custom functions and statements (an example of that is the `Useless.function`).

Expanding R’s capabilities by installing additional function packages

The R language includes an extensive array of functions. Those are known as base R, or core R functions. However, sometimes developers will write R functions that enhance R’s capabilities beyond the scope of the basic core functions (e.g. enhanced graphical capabilities, faster and more efficient data manipulation, etc.) or functions designed for a specialized task which doesn’t concern most R users (e.g. population genetics, SQL integration etc.).

Usually developers package their functions in what is known as function libraries, or packages, and publish them to the public. These packages can be downloaded and installed by R users, and the functions they contain expand the capabilities of R.

There are several repositories that store R packages, here we will go over 3 of them.

1. The CRAN repository

The CRAN repository is the main repository for R packages. It is maintained and rigorously curated by the R foundation and currently contains over 15K packages. The scope of these packages is very broad and they can enhance

R capabilities in anything from graphical representation of data, to genomic analysis, to development of web applications. Each of the packages featured in the CRAN repository contains documentations including a manual with a list of all functions included in the package and examples for how to use them. Some contain workflows.

To install a package from the CRAN repository use the function `install.packages`, note that `install.packages` doesn't load the package. If you want to call functions from within this package you need to load it using the function `library()`. Alternatively you can use `::` which allows you to call the function without load the package that contains it. This may also be used if you loaded two packages that contain a function with similar name, or if a function from one package is masking another.

```
> #note the use of '' around the argument in these functions, they will recognize the  
>  
> install.packages('x')  
> library('x')  
> x::function.x()
```

2. Bioconductor:

Bioconductor is a repository for packages designed for handle and analyse high throughput biological data (e.g. omics, flow cytometry, etc.). Contains over 1800 packages, however not all packages aimed at analyzing high throughput biological data are published in Bioconductor since it requires the variables to be stored in an object with specialized structure (S4 object). The packages are curated by the Bioconductor project for open source bioinformatic software. Each Bioconductor package contains at least one vignette, a document that provides a task-oriented description of package functionality. Most also contains a manual with example SOPs.

3. GitHub:

GitHub is basically a developer's domain, and most if not all R packages are stored on GitHub at one point or another during the stage for their development. Most packages also have a GitHub version in addition to the CRAN or Bioconductor version. The version on GitHub is a developer version with capabilities absent from the official distribution of the package. Those versions are normally less stable as they are in the testing stage of the development. There is no curation of github packages, therefore documentation is patchy.

Installing packages from Bioconductor and GitHub require additional package managers and have their own special installation functions. We will go over it during labs 3-4.

Quick R exercise - installing and loading packages

1. Install the package `vegan` from the CRAN repository
 2. What is the purpose of this package?
 3. How many arguments does the `anosim` function from this package accept, how many of them are variables?
-

Variable assignment in R (<- vs -> vs =)

R has three ways of assigning values to variables, two of them are leftwards and one rightwards. Leftwards assignment is the most common way to assign value to variables, this is similar to other programming language like python or C++. In R this is achieved by using the assign operator `<-`, or alternatively similar to python and C++ by using the equal operator `=`.

Leftwards assignment groups the values from right to left, therefore in the snippet below the value of the variable `v` will be 1 and the value of variable `v2` will also be 1 since it will get the value of variable `v`.

While on the surface both methods work identically, beneath the surface they are quite different. `=` is primarily used to assign value to function arguments and using it out of context may lead to unforeseen errors. Therefore, the convention is to use `<-` over `=`.

```
> #This is leftwards assignment
> v <- 1
>
> #This is an alternative leftwards assignment
>
> v2 = v
>
> v
```

```
[1] 1
```

```
> v2
```

```
[1] 1
```

Rightwards assignment works opposite to leftwards assignment. It uses the rightwards assign operator `->`, and groups everything left to right. Therefore, in the code snippet below, the value of variable `v2` will now be 3. Note that rightwards assignment is discouraged and good coding practices dictate that all variables should be assigned using the leftwards method.

```
> 3 -> v2
>
> v
```

```
[1] 1
```

```
> v2
```

```
[1] 3
```

A note on aliasing and R

The code above assigns the value of `v` to variable `v2`. Unlike python or java for example, this does not create an alias (creating an alternative name for a variable stored in memory) and there is no link between `v` and `v2`. Therefore, `v2` can be modified without the risk of modifying the variable `v` as well.

Rules for variable naming in R

Variable naming in R follows a few simple rules:

- Variable names must not be the same as R's special characters or functions (except for `c`)
- Variable names must always start with a letter, however they may contain numbers and other characters
- Special characters must be escaped using `"`, and it is recommended that they be avoided in general
- Variable names are case sensitive (but you already know that)

Data types (objects) in R

The R language has multiple classes of data types often referred to as objects, though R is not an object oriented programming language per se. Those include but not limited to vectors, lists, matrices, and data.frames. Each of these is optimized to handle either single or multidimensional information and is part of what makes R such a powerful tool in handling, analyzing, and representing data. This part will go over the commonly used data types in R.

“Atomic” vectors

Atomic vectors are probably the most basic form of data class in R. Similar to lists in python, they contain a sequence of elements. In fact, the output from all the functions used in this tutorial so far were vectors with one element. That is the reason `[1]` was displayed next to the output, to denote the first element in the vector. In R, each atomic vector class is capable of only storing information of the same type. There are four main classes of atomic vectors in R, this part will detail them and how to work with them.

Numeric vectors (double and integer)

Numeric vectors store elements that are numbers, R has two common classes of numeric vectors: doubles (double precision floating point numbers) and integers.

Let's make some:

```
> dbl <- 1.5
> dbl
```

```
[1] 1.5
```

```
> int <- 1L
> int
```

```
[1] 1
```

Ok, that was cheating. What if you actually want to create a vector with multiple elements? The easiest way to do that (or any other type of atomic vector) is using the function `c()` which you have already encountered.

```
> #double
> dub.vec <- c(1,2,3)
>
> #integer
> int.vec <- c(1L,2L,3L)
>
> #alt way to get an integer sequence, : tells R to get all natural numbers between 1 and 3
>
> int.vec2 <- (1:3)
>
>
> dub.vec
```

```
[1] 1 2 3
```

```
> int.vec
```

```
[1] 1 2 3
```

```
> int.vec2
```

```
[1] 1 2 3
```

```
> #Note this is one way of comparing variables in R
> dub.vec == int.vec
```

```
[1] TRUE TRUE TRUE
```

The snippet above uses natural numbers, the output of the two vectors is pretty much identical (as demonstrated by the `is.equal` operator `==`). Furthermore, as will be detailed later, regardless of the type of numbers used, R will automatically convert classes of numeric vectors when needed. However, sometimes

it might behoove you know whether a vector contains floats or integers. The RStudio global environment pane will denote if a vector is a number or integer, however a more R way to do it is to use the function `typeof()`.

```
> typeof(dub.vec)
```

```
[1] "double"
```

```
> typeof(int.vec)
```

```
[1] "integer"
```

Working with numerical vectors is straightforward, R allows you to perform mathematical operations on them, sum them and even concatenate them. When concatenating an integer and double to a single vector R will coerce it to double. This is because coercing a float to integer will result in dropping the float (i.e. 4.99 will be coerced to 4 you can try it using `as.integer(4.99)`). Therefore coercing to double is less restrictive.

```
> #concatenating vectors plus a bonus example of nesting functions.
```

```
> #note that you can use the assign operator within a function, that's one of the diff
```

```
> #between it and using "=" .
```

```
> typeof(num.vec <- c(dub.vec,int.vec))
```

```
[1] "double"
```

```
> num.vec
```

```
[1] 1 2 3 1 2 3
```

```
> #note how when performing a mathematical operation on a vector it is performed indivi
```

```
> #the elements of the vector, negating the need to use a for loop. Flanking the oper
```

```
>
```

```
> (sq.num.vec <- num.vec^2)
```

```
[1] 1 4 9 1 4 9
```

Indexing and subsetting numeric vectors R makes it easy to index, access and subset every element of a numeric vector using the square brackets `[]`. One thing to note is that unlike other programming language (looking at you python), R starts indexing at 1 and not at 0.

In the example below, I used the function `rnorm()` to generate a vector of 100 random numbers normally distributed around a mean of 45 (which would probably be this course's average) and a standard deviation of 18. Note that I didn't list the arguments for `rnorm()`, if arguments are not specified R assumes your input is in the order defaulted by the function you call.

```
> #Note how I didn't specify the arguments, R knows to match them
```

```
> #you just have to make sure you write them in the correct order.
```

```
> this.course.average <- round(rnorm(100, 45, 18),2)
```

```
>
> #We can see if our vector is indeed 100 elements long using the function length
>
> length(this.course.avarage)
```

```
[1] 100
```

```
> #Head lets you glimpse the first n elements in a variable, default is 6 (tail is for last)
> head(this.course.avarage)
```

```
[1] 55.63 70.74 46.14 54.55 37.09 18.52
```

OK, you made a vector now what? As mentioned above, elements along the vector can be accessed (sometimes referred to as indexed) using the []. Vectors are mutable meaning that accessing elements allows you to modify them. The following code is an example of how.

```
> #let's access the first 10 elements in the vector.
> this.course.avarage[1:10]
```

```
[1] 55.63 70.74 46.14 54.55 37.09 18.52 75.74 53.68 37.29 47.65
```

```
> #You can also replace them with whatever it is you want.
> #Let's replace the first 10 elements with a copy of the last 10 elements.
>
> this.course.avarage[1:10] <- this.course.avarage[91:100]
>
> #Did it work?
>
> this.course.avarage[1:10]
```

```
[1] 41.51 58.84 42.42 37.44 73.20 33.47 58.75 26.77 30.44 56.61
```

```
> this.course.avarage[91:100]
```

```
[1] 41.51 58.84 42.42 37.44 73.20 33.47 58.75 26.77 30.44 56.61
```

```
> #Did we replace or just increased the size of the vector?
> length(this.course.avarage)
```

```
[1] 100
```

Let's look at another example of indexing, were we subset a vector in reverse and modify using some conditions without resorting to conditional statements.

```
> #note that indexing and subsetting works in reverse order too
> another.course.avarage <- c(this.course.avarage[1:5],this.course.avarage[25:20])
>
> #What would the length of this vector be?
>
```

```
> #Another great aspect of vectors is that they allow you to change only a subset values without
```

```

>
> #Let's create a new vector
>
> another.course.avarage2 <- rnorm(n=100,mean=75,sd=5)
>
> #Let's access only marks higher than the avarage
> another.course.avarage2[another.course.avarage2>75]

[1] 77.86762 83.67366 77.63294 77.49356 77.64779 80.49451 75.60915 84.27786
[9] 80.15926 79.93986 78.59264 82.41114 78.60995 76.54949 77.42665 75.35348
[17] 76.30133 79.13636 80.85885 78.36407 77.96290 76.65210 80.75706 80.90959
[25] 82.90016 82.49072 75.03349 75.61420 77.27623 82.99201 78.59076 79.79772
[33] 76.88908 77.56748 81.05216 76.25810 85.80128 75.69248 78.32324 84.80004
[41] 80.22270 75.61953 79.65478 78.61940 79.54729 76.13486 81.10263

> #lets change anything equal to or grater than the 75 percentile to 100
> #this can be done using the quantile function and subsetting the 3 element of the vec
> another.course.avarage2[another.course.avarage2 >= quantile(another.course.avarage2)
>
> another.course.avarage2[another.course.avarage2==100]

[1] 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100
[20] 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100
[39] 100 100 100 100 100 100 100 100 100 100 100 100

```

Quick note on reproducibility

Note that the numbers generated by the code snippet above will change every time you run it. This is due to the way random numbers (more accurately pseudorandom numbers) are generated by “random” number generating algorithms. If you want to make sure that your code is reproducible you need to set the seed that initializes the pseudorandom number generating algorithm rather than allowing it to be set randomly. You can do it by incorporating the function `set.seed()` before you generate numbers, you can use any integer between the brackets.

Quick R exercise - indexing and subsetting:

1. Use the function `seq` to create a numeric vector of all the natural numbers between 1 and 10. If you are unsure of how to use `seq`, use the help function.
2. What type of a numeric vector is the vector you created?

3. Create another numeric vector of 10 random elements between 1 and 10 using the function `runif()`
4. What type of a numeric vector did it create?
5. Create a third vector by multiplying your first vector by the second vector
6. Call the new vector, what can you tell of the order of operations when multiplying vectors in R?
7. Replace the 7th element of the first vector, the 8th element of the second vector and the 1st element of the third vector.
8. Create a new vector with the final element being -8, the first element being the 7th element of the third vector, the second element being the 5th element of the first vector and the third element being the first element of the

Character vectors:

Character vectors only store elements that are characters. These could be letters, numbers, or characters. In order for any of those to count as a character they need to be flanked by double quotation mark (e.g. "j").

```
> #Here is how you create a character vector
> char.vec <- c("a", "abc", "abcd")
>
> #Note the length of this vector is 4, the length of each string of characters doesn't impact the
> #length of the vector
>
> length(char.vec)
```

```
[1] 3
```

```
> #Character vectors can contain numbers too
> char.vec2 <- c("1", "2", "3")
> typeof(char.vec2)
```

```
[1] "character"
```

```
> #Tip, R can convert(coerce) this to a numeric vector
> #There is an as.x function for almost every data type in R
>  #(e.g. as.numeric, as.double, as.character, etc.)
> char.vec2b <- as.numeric(char.vec2)
>
> #When concatenating a character and a numeric vector, R will coerce the numeric vector to character
> char.vec3 <- c(char.vec2, int.vec)
> typeof(char.vec3)
```

```
[1] "character"
```

```
> #alternatively you can use the class, class will not differentiate
> #between the different numeric vectors but will between the different types of vectors
> class(char.vec3)
```

```
[1] "character"
```

```
> #If you need to generate a sequence of letters from a-z
> char.vec4 <- letters[1:7]
```

Indexing character vectors is done similarly to numeric vectors.

```
> char.vec[1]
```

```
[1] "a"
```

```
> #replacing the first element
> char.vec[1] <- "aa"
>
> char.vec[1]
```

```
[1] "aa"
```

```
> #adding a fourth element
> char.vec[4] <- "Fourth"
>
> char.vec[4]
```

```
[1] "Fourth"
```

```
> #To count the number of characters in each of the strings you can use the function nchar
> nchar(char.vec)
```

```
[1] 2 3 4 6
```

```
> #just like with numerical vectors it is possible to subset by value
> char.vec[char.vec=="a"]
```

```
character(0)
```

```
> #or replace a value
> char.vec[char.vec=="a"] <- "a2"
```

####Quick R exercise - character vectors:{-}

1. Use R to calculate the total number of character in char.vec (avoid hard coding)
2. Use indexing to change the 4th character in the string occupying the 4th element in the vector char.vec with the letter “B”, no hard coding. Did it work?

A note on working with strings in R

As you notice in the exercise, indexing a character vector is not the same as indexing the strings that make each element of the vector. This does not mean that strings are immutable, R has a lot of tools in its arsenal geared towards manipulating strings. While the future labs will deal with handling biological strings, this is a good opportunity to look at how R handles strings in general. Base R has functions such as `grep`, `substring` and `gsub` that work the same way `grep` in bash works, however, I find the `stringr` package much more useful and intuitive. You can download a cheatsheet guide to `stringr` [here](#)

Lets look at an example for the capabilities of `stringr`

```
> #load the package you can install it from the cran repo with install.packages
> library(stringr)
> boring <- "It was the best of times it was the worst of time"
>
> #count the number of characters
> str_count(boring)
```

```
[1] 49
```

```
> #split the string in boring to several elements after every space
> #the indexing here is for a list, we will discuss lists later in this lab
>
> boring2 <- str_split(boring, " ")[[1]] #note how white space is referred to
>
> #replace a part of a string, lets correct the typo at the end of the sentence to times
>
> boring <- str_replace(boring, "time$", "times") #$ is a regular expression that matches the e
> boring
```

```
[1] "It was the best of times it was the worst of times"
```

```
> #How many times does the word it appear in the string?
> #Note that R is case sensitive, (?i) searches in a case insensitive way (another regular expres
>
> str_count(boring, "(?i)it")
```

```
[1] 2
```

Quick R exercise - strings: Now lets do something a bit biological Download the stringr cheatsheet and run the following code that will generate a string of 100 nucleotides.

```
> #four bases
> bases <- c("A","T","G","C")
>
> #note seting seeds
> set.seed(12)
>
> #Using sample let's generate a random sequence of 10000 nucleotides, slightly larger
> dna <- paste(sample(bases,10000, replace = T), collapse = "")
```

1. Use stringr to calculate the %GC of this string
 2. Use stringr to find out how many times do the codons ATG and TAG appear in the string?
-

Logical vectors

Logical vectors (or Boolean vectors) contain binary elements, i.e. true or false or 1 and 0. They are the result of logical operators that compare variables. Due to their binary nature, R treats them like numeric vectors (double to be exact). Logical vectors can be indexed as any other vector. The example snippet bellow shows the result of a logical operator on the vector `this.class.average`. The operator will evaluate each element of the vector and determine whether this element is larger than 55. If the element is larger than the result will be TRUE and if it is not the result will be false.

```
> log.vec <- this.course.avarage > 55
> log.vec
```

```
[1] FALSE TRUE FALSE FALSE TRUE FALSE TRUE FALSE FALSE TRUE FALSE TRUE
[13] FALSE FALSE TRUE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[25] FALSE TRUE FALSE TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[37] FALSE FALSE FALSE FALSE TRUE FALSE TRUE FALSE FALSE FALSE TRUE FALSE
[49] FALSE FALSE FALSE TRUE TRUE FALSE TRUE TRUE TRUE FALSE FALSE TRUE
[61] FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE TRUE FALSE FALSE FALSE
[73] TRUE FALSE TRUE FALSE TRUE FALSE TRUE TRUE FALSE FALSE FALSE FALSE
[85] TRUE FALSE TRUE FALSE FALSE TRUE FALSE TRUE FALSE FALSE TRUE FALSE
[97] TRUE FALSE FALSE TRUE
```

You can use logical vectors to index and subset types of vectors using the `which` function.

```
> #The which function tells you the position of elements fulfill a condition (give an
> head(which(this.course.avarage > 50))
```

```
[1] 2 5 7 10 12 15
```

```
> #Can use it to subset only the elements that do
> #Here, asking R to subset only the elements larger than 50
> passed.the.course <- this.course.avarage[which(this.course.avarage > 50)]
>
> length(passed.the.course)
```

```
[1] 42
```

```
> #of course you can also use the subset function do achieve the same results
>
> passed.the.course2 <- subset(this.course.avarage, this.course.avarage >50)
> length(passed.the.course2)
```

```
[1] 42
```

```
> #Lets compare the two vectors using a logical operator, the result of this will be a logical vector
> passed.the.course == passed.the.course2
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
> #alternatively you can ask R which positions between the two vectors are not the same
> which(passed.the.course != passed.the.course2)
```

```
integer(0)
```

Checking and coercing

Before we continue with data types in R, lets have a look at how we can tell which is which and how do we convert (coerce) variable from one data type to another.

The class function

The `class()` function lets you know what class of data type does a particular variable belong to. It is useful since some packages (especially those from bio-conductor) import data as variables containing more than one type of objects.

```
> class(dna)
```

```
[1] "character"
```

The is.x function family

The `is.x` function family is another option to query for the type of a variable. The output of any of these functions is a logical vector with the length of 1.

```
> #is a variable an atomic vector?  
> is.atomic(passed.the.course)
```

```
[1] TRUE
```

```
> #Note logical vectors are also numeric  
> is.numeric(passed.the.course)
```

```
[1] TRUE
```

```
> is.character(passed.the.course)
```

```
[1] FALSE
```

```
> is.logical(passed.the.course)
```

```
[1] FALSE
```

```
> typeof(passed.the.course)
```

```
[1] "double"
```

The as.x function family

R variables can be coerced to another data type, depending of restriction. This can be done using an as.x function. There is an as.x for every data type in R. Here are some examples

```
> numbers <- seq(1,100, by = 5)  
> typeof(numbers)
```

```
[1] "double"
```

```
> class(numbers)
```

```
[1] "numeric"
```

```
> numbers <- as.character(numbers)  
> class(numbers)
```

```
[1] "character"
```

```
> #lets see what happens when a character vector is coerced to numeric  
> lets <- letters[1:5]
```

Quick R exercise - checking and coercing

1. Try coercing numbers back to a numeric vector and then use the sum function to sum it, is it possible?

2. Create a logical vector of 6 elements and coerce it to an integer vector, what happens
 3. Create sequence of 10 numbers from 3.5-4.5, coerce the double vector to an integer, what happened?
 4. What is the difference between `is.numeric` and `is.integer`?
-

Factors - vectors' evil cousins

Factors are special case of integer vectors containing only predefined values. Factors are comprised of two value sets, the elements of the vector and the levels. Levels are the defined set of values (or categories) allowed in the vector. While factors are useful for certain type of tasks, e.g. when you know exactly what value categories are expected in a data-set, often times this is not the case. When working with R, factors are actually pretty common since most of BaseR data importation functions coerce vectors into factors, therefore it is worth while to dedicated a few minute to familiarize yourself with them.

The snippet bellow includes a data-set of the quality of each season of game of thrones. There are three possible values

```
> #Let's create a character vector with 8 elements
>
> b <- c(rep("Terrible",7), "Thank god that series is over")
>
> #Coerce it into a factor
> seasons_of_GoT <- factor(b, levels = c("Terrible", "Thank god that series is over", "Great"))
>
> seasons_of_GoT
```

```
[1] Terrible           Terrible
[3] Terrible           Terrible
[5] Terrible           Terrible
[7] Terrible           Thank god that series is over
Levels: Terrible Thank god that series is over Great
```

```
> typeof(seasons_of_GoT)
```

```
[1] "integer"
```

```
> class(seasons_of_GoT)
```

```
[1] "factor"
```

```
> #To edit the levels of a factor use the level function, e.g. great is not a real option for a
>
> levels(seasons_of_GoT) <- c("Terrible", "Thank god that series is over", "Extra terrible")
```

It is worth noting the while factors are integers the values they contain are treated as characters. When coercing a factor back to an atomic vector, it is necessary to coerce it first to a character. Otherwise the vector will contain the number of available levels rather than the elements of the factor.

```
> a <- seq(5, 15, by = 1)
>
> a2 <- factor(a, levels = seq(5, 19, by = 1))
>
> typeof(a)
```

```
[1] "double"
```

```
> typeof(a2)
```

```
[1] "integer"
```

```
> sum(a2)
```

```
Error in Summary.factor(structure(1:11, levels = c("5", "6", "7", "8", : 'sum' not mean
```

```
> sum(a)
```

```
[1] 110
```

```
> #to coerce the factor to a numeric vector you have to pass through as.character otherwise
>
> a3 <- as.numeric(a2)
> a4 <- as.numeric(as.character(a2))
>
> #level 1-11
> head(a3,4)
```

```
[1] 1 2 3 4
```

```
> #elements 1-11
```

```
> head(a4,4)
```

```
[1] 5 6 7 8
```

```
> a==a4
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

Matrices

Matrices are two dimensional vectors, which is a fancy way of saying that a matrix is a tabulated vector. Because a matrix is essentially a linear vector folded into a table, matrices only contain elements of the same data type (e.g. only integer, only characters, etc).

The code snippet below uses the function `dim` to change the dimensions of the vector.

```
> mat <- seq(3.5, 4.6, by = 0.1)
>
> dim(mat)
```

NULL

```
> #Coerce the vector into a matrix by changing its dimensions (i.e. adding columns and rows)
> #When changing dimensions the order is rows, columns (here 4 rows, 3 columns)
> dim(mat) <- c(4,3)
>
> mat
```

```
      [,1] [,2] [,3]
[1,]  3.5  3.9  4.3
[2,]  3.6  4.0  4.4
[3,]  3.7  4.1  4.5
[4,]  3.8  4.2  4.6
```

Note how the vector is folded into a two dimensional vector from top to bottom in each column. This essentially converts each column and each row to a vector.

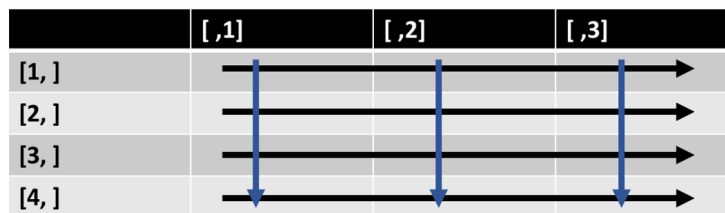


Figure 4: R matrix, each row and each column are vectors
(#fig:mi)

We can also use the `class` and `is.x` function to inspect the column and the matrix as a whole and see that it is indeed both a vector and made of vectors.

```
> class(mat)
```

```
[1] "matrix" "array"
```

```
> class(mat[1,])
```

```
[1] "numeric"
```

```
> class(mat[,1])
```

```
[1] "numeric"
```

```
> is.numeric(mat)
```

```
[1] TRUE
```

```
> is.atomic(mat)
```

```
[1] TRUE
```

```
> is.atomic(mat[1,])
```

```
[1] TRUE
```

Quick R exercise - matrices 1

There are several R functions that can be used to generate a matrix.

1. Create a numeric vector of 10 elements, use the matrix function to convert it to a matrix with 5 rows and two columns, what happens when the argument `byrow` is set to TRUE?
 2. Create a character vector of 10 element use the function `cbind()` to bind it with the vector from the previous question. Are the vectors columns or rows? What happens when you use the function `rbind()`?
 3. Use the `typeof()` function on one of the matrices from question 2, what type of data does this matrix contain, what does it tell you about the coercion of data type within a matrix?
-

Working with matrices

A matrix is basically a vector with two dimensions when a matrix is indexed like a vector, i.e. using `[]`. By default, when a matrix is indexed using `[]` R will return the value found on the *n*th position of the original vector (see snippet below). One of R strength is that it allows to also index and access any combination of columns and rows, not just individual elements. This is done similar to accessing a vector only the positions of the rows and columns are separated by a comma, i.e. `matrix[row,column]`. This form of indexing allows to subset specific rows and columns from the matrix, the following snippet demonstrates how.

```
> #Mat[5] will output the element occupying the fifth position of the vector used to c
> #matrix i.e. 3.9
> mat[5]
```

```
[1] 3.9
```

```
> #Mat[,1] will output the first column of the matrix
> mat[,1]
```

```
[1] 3.5 3.6 3.7 3.8
```

```
> #mat[,2:3] will output the 2nd and 3rd columns of the matrix
>
> mat[,2:3]
```

```
      [,1] [,2]
[1,]  3.9  4.3
[2,]  4.0  4.4
[3,]  4.1  4.5
[4,]  4.2  4.6
```

```
> #mat[1,] will output the first row of the matrix
> mat[1,]
```

```
[1] 3.5 3.9 4.3
```

```
> #mat[1:2,] will output the first 2 rows of the matrix
> mat[1:2,]
```

```
      [,1] [,2] [,3]
[1,]  3.5  3.9  4.3
[2,]  3.6  4.0  4.4
```

```
> #mat[1:2,1:2] will subset the first two rows and columns of the matrix
> mat[1:2,1:2]
```

```
      [,1] [,2]
[1,]  3.5  3.9
[2,]  3.6  4.0
```

It is possible to name the rows and columns of a matrix, once named the names can be used to index and subset the matrix. The code snippet below shows how to achieve this using the `colnames()` and `rownames()` functions.

```
> #The rows and columns of a matrix can be named, so that the vectors have meaningful names.
> #This is done using the functions colnames and rownames
>
> stud_mrks <- mat
> rn <- c("Class1", "Class2", "Class3", "Class4")
> cn <- c("Student1", "Student2", "Student3")
> rownames(stud_mrks) <- rn
> colnames(stud_mrks) <- cn
>
> stud_mrks
```

	Student1	Student2	Student3
Class1	3.5	3.9	4.3
Class2	3.6	4.0	4.4
Class3	3.7	4.1	4.5

```
Class4      3.8      4.2      4.6
```

```
> #The names can be used to index the matrix, note the use of comma.
> stud_mrks["Class1",]
```

```
Student1 Student2 Student3
      3.5      3.9      4.3
```

```
> stud_mrks[, "Student1"]
```

```
Class1 Class2 Class3 Class4
      3.5      3.6      3.7      3.8
```

Quick R exercise - matrices 2

1. Assign the matrix mat to the variable mat_bs and replace the first row of mat_bs with A,B,C
2. Replace the 4th element in the matrix mat_bs with 1000
3. Assign the 3rd column of the matrix mat_bs the the vector v_bs
4. Use the function t to transform the matrix mat_bs, how many rows does it have now?

One of R's greatest strength is the ability to perform tasks on a matrix as a whole, negating the need to use a loop every time an operation is needed to be done on several columns.

```
> #Let's generate a new matrix, this time with matrix function
> #byrow, fill the matrix by rows
> mat2 <- matrix(sample(1:1000,100,replace=F),nrow = 10,ncol = 10,byrow = T)
> head(mat2)
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,]  859  449  110  675  232  893  619  974  271  630
[2,]  531  621  753  366  157  866  135  705   21  769
[3,]  886  990   98  823  586  411  150  783  776  382
[4,]  937  652  540  327  239  741  951   46  681  842
[5,]  903  670  947  446  785  661   53  646   12  113
[6,]   25  538  433  763  227  622   71  173  521  762
```

```
> #When handling data, there is sometimes a need to transform it.
> #Log transformation is a common one. R's handling of matrices allows to do it without
> #Note that I am copying the matrix because I still want to use the original
> mat2_log <- log10(mat2)
> head(mat2_log)
```

```

      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]
[1,] 2.933993 2.652246 2.041393 2.829304 2.365488 2.950851 2.791691 2.988559
[2,] 2.725095 2.793092 2.876795 2.563481 2.195900 2.937518 2.130334 2.848189
[3,] 2.947434 2.995635 1.991226 2.915400 2.767898 2.613842 2.176091 2.893762
[4,] 2.971740 2.814248 2.732394 2.514548 2.378398 2.869818 2.978181 1.662758
[5,] 2.955688 2.826075 2.976350 2.649335 2.894870 2.820201 1.724276 2.810233
[6,] 1.397940 2.730782 2.636488 2.882525 2.356026 2.793790 1.851258 2.238046
      [,9]      [,10]
[1,] 2.432969 2.799341
[2,] 1.322219 2.885926
[3,] 2.889862 2.582063
[4,] 2.833147 2.925312
[5,] 1.079181 2.053078
[6,] 2.716838 2.881955

```

```

> #Another common transformation is to calculate the proportion of value in the column or row (of)
> #All that's needed to do is to divide the matrix by the sum of its columns or rows
>
> mat2_freq_row <- mat2/rowSums(mat2)
> head(mat2_freq_row)

```

```

      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
[1,] 0.150385154 0.07860644 0.01925770 0.11817227 0.04061625 0.15633754
[2,] 0.107839155 0.12611698 0.15292445 0.07432981 0.03188465 0.17587327
[3,] 0.150552251 0.16822430 0.01665251 0.13984707 0.09957519 0.06983857
[4,] 0.157320349 0.10946944 0.09066488 0.05490262 0.04012760 0.12441236
[5,] 0.172459893 0.12796028 0.18086325 0.08517953 0.14992361 0.12624141
[6,] 0.006045949 0.13010883 0.10471584 0.18452237 0.05489722 0.15042322
      [,7]      [,8]      [,9]      [,10]
[1,] 0.10836835 0.170518207 0.047443978 0.11029412
[2,] 0.02741673 0.143176279 0.004264825 0.15617384
[3,] 0.02548853 0.133050127 0.131860663 0.06491079
[4,] 0.15967092 0.007723304 0.114338482 0.14137005
[5,] 0.01012223 0.123376623 0.002291826 0.02158136
[6,] 0.01717050 0.041837969 0.125997582 0.18428053

```

```

> #rowSums should be 1
> rowSums(mat2_freq_row)

```

```

[1] 1 1 1 1 1 1 1 1 1 1

```

```

> #When normalizing by column it is impossible to just use the "/" operator, due to the way R stores
> #use ?sweep to see which each of the arguments means.
>
> mat2_freq_columns <- sweep(mat2,2,colSums(mat2),"/")
> head(mat2_freq_columns)

```

```

      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]

```

```
[1,] 0.141492341 0.07405575 0.01941405 0.12525515 0.04458109 0.11777895
[2,] 0.087464998 0.10242454 0.13289799 0.06791613 0.03016910 0.11421788
[3,] 0.145939713 0.16328550 0.01729615 0.15271850 0.11260569 0.05420733
[4,] 0.154340306 0.10753752 0.09530533 0.06067916 0.04592621 0.09773147
[5,] 0.148739911 0.11050635 0.16713731 0.08276118 0.15084550 0.08718016
[6,] 0.004117938 0.08873495 0.07642076 0.14158471 0.04362029 0.08203640
      [,7]      [,8]      [,9]     [,10]
[1,] 0.16027965 0.204407135 0.057684121 0.10607846
[2,] 0.03495598 0.147953830 0.004469987 0.12948308
[3,] 0.03883998 0.164323190 0.165176671 0.06432059
[4,] 0.24624547 0.009653725 0.144955300 0.14177471
[5,] 0.01372346 0.135571878 0.002554278 0.01902677
[6,] 0.01838426 0.036306401 0.110898255 0.12830443
```

```
> #If everything worked out the sum of each column should be 1 now
> colSums(mat2_freq_columns)
```

```
[1] 1 1 1 1 1 1 1 1 1 1
```

```
> #Just like vectors it is also possible to preform mathematical operations on two matrices
> mat2_mlt <- mat2_freq_columns*mat2_log
> head(mat2_mlt)
```

```
      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
[1,] 0.41513756 0.1964141 0.03963170 0.3543849 0.10545604 0.3475482 0.44745119
[2,] 0.23835039 0.2860811 0.38232026 0.1741017 0.06624832 0.3355171 0.07446791
[3,] 0.43014763 0.4891438 0.03444055 0.4452355 0.31168102 0.1416894 0.08451934
[4,] 0.45865920 0.3026372 0.26041169 0.1525806 0.10923080 0.2804716 0.73336346
[5,] 0.43962873 0.3122992 0.49745913 0.2192621 0.43667807 0.2458656 0.02366303
[6,] 0.00575663 0.2423158 0.20148240 0.4081214 0.10277054 0.2291925 0.03403401
      [,8]      [,9]     [,10]
[1,] 0.61088278 0.140343695 0.29694975
[2,] 0.42140049 0.005910303 0.37367863
[3,] 0.47551216 0.477337739 0.16607985
[4,] 0.01605181 0.410679690 0.41473527
[5,] 0.38098850 0.002756529 0.03906346
[6,] 0.08125540 0.301292561 0.36976759
```

Because matrices are tables but behave like vectors, operating on them is memory efficient and many R packages use them as a preferred object.

Lists

A list is a multidimensional vector, which is a fancy way of saying that a list is a vector that can house any R object as an element (e.g. other vectors, matrices and even other lists). The elements of a list are independent of each other, making it a great container to store the output of functions that handle data. For example, some functions from the `stringr` package output their result as

lists, especially those with `_all` suffix.

The code snippet bellow demonstrate how to create lists, as always there is more than one way to obtain your goal.

```
> #creating an empty list of 10 elements
> #Because a list is basically a vector of elements, it is possible to creat it using the vector
> list1 <- vector(mode = "list", length = 10)
> class(list1)
```

```
[1] "list"
```

```
> is.vector(list1)
```

```
[1] TRUE
```

```
> is.atomic(list1) #why is it not atomic?
```

```
[1] FALSE
```

```
> head(list1,3)
```

```
[[1]]
```

```
NULL
```

```
[[2]]
```

```
NULL
```

```
[[3]]
```

```
NULL
```

```
> #Creating a list by using the list function
> list2 <- list(matrix(sample(10:1000,100,replace = F),ncol = 2),
+               cbind(seq(1,100,by=2),rep(10,50)),c("Category1","Category2"),list1,
+               factor(c("Cat","Dog","Cat","Mouse"))) )
```

Working with lists

The previous snippet demonstrated that lists can store any type of R object, even other lists. As mentioned, the elements of a list are independent of each other and in order to modify them they need to be accessed individually. Since lists are vectors, each element can be inspected, like one would inspect a vector.

```
> #Inspecting the 3rd element of list2
> list2[3]
```

```
[[1]]
```

```
[1] "Category1" "Category2"
```

However, indexing a list like a vector does not allow you to modify the element

```
> #Note how the head function is unable to truncate the element to its first 2 components
> head(list2[5],2)
```

```
[[1]]
[1] Cat   Dog   Cat   Mouse
Levels: Cat Dog Mouse
```

```
> #Note how indexing like a vector does not allow you to subset the element itself
> list2[5][1]
```

```
[[1]]
[1] Cat   Dog   Cat   Mouse
Levels: Cat Dog Mouse
```

In case you noticed, when using the `head()` function on a list each element is surrounded by double square brackets (`[[`). When subsetting a list with `[`, it will always return another list. Think of it elements one on a character vector where each element is a string. You can see each string even replace them but can't modify them. In order to access and modify an element of the list `[[` have to be used.

```
> #Subsetting the last element of list2 and assigning it to the first element of list1
> list1[[1]] <- list2[[5]][1:3]
> list1[1]
```

```
[[1]]
[1] Cat Dog Cat
Levels: Cat Dog Mouse
```

Lists and the `$` operator

`$` is an operator that can be thought of as the equivalent of `[[ ]]`, therefore it can be used to access elements of a list. However, in order to use it the elements must be named first.

```
> #Are the elements of list2 named?
> names(list2)
```

```
NULL
```

```
> #Naming the elements of the list
> names(list2) <- c("Sample1","Sample2","Categories","Nested_list","Pets")
>
> names(list2)
```

```
[1] "Sample1"      "Sample2"      "Categories"    "Nested_list"  "Pets"
```

```
> #Accessing the list using $, note how rstudio autocompletes the elements
> list2$Categories
```

```
[1] "Category1" "Category2"
```

```
> list2$Categories[2]
```

```
[1] "Category2"
```

```
> list2$Sample2[1,2]
```

```
[1] 10
```

```
> list2$Sample2[1,2] <- 11
```

```
> list2$Sample2[1,2]
```

```
[1] 11
```

Quick R exercise - lists

Let's wrap up working with lists with the following exercise:

1. Change the values of the 10th row in sample2 to 7 and 5
2. Log10 transform Sample2 and assign it to the first element of list1
3. Name the first element of list1 "S2_log"
4. Use sweep to normalize Sample1 by dividing it by the sum of its columns, assign it to the 2nd element of list1 and name is S1_norm
5. Create a list called list3 that contains only the first 2 elements of list1

Data frames

Data frames are a commonly used objects in R, especially in data preparation and analysis. Data frames are a special case of a list containing vectors of equal length as element, giving it a rectangular shape. The vectors make up the columns of the data frame.

Let's create a data frame or two

```
> # data frames can be created by using the data.frame function
```

```
> #Note that by default strings are coerced to factors, which can be annoying
```

```
> #Can avoid be using stringsAsFactors=FALSE
```

```
> df1 <- data.frame(X = seq(1,10, by = 1),
+                  Y = seq(100,1000, by = 100),
+                  Z = LETTERS[1:10], stringsAsFactors = FALSE)
>
```

```
> #Each column is an independent vector, that's why columns X and Y can be numeric and Z a character
```

```
> str(df1)
```

```
'data.frame': 10 obs. of 3 variables:
```

```
$ X: num 1 2 3 4 5 6 7 8 9 10
```

```
$ Y: num 100 200 300 400 500 600 700 800 900 1000
```

```
$ Z: chr "A" "B" "C" "D" ...
> # It is also possible to coerce a matrix to a data frame and vice versa
>
> df2 <- as.data.frame(mat2)
>
> #let's see what a data frame looks like
> head(df1,4)
```

```
   X   Y Z
1 1 100 A
2 2 200 B
3 3 300 C
4 4 400 D
```

```
> head(df2,4)
```

```
   V1  V2  V3  V4  V5  V6  V7  V8  V9 V10
1 859 449 110 675 232 893 619 974 271 630
2 531 621 753 366 157 866 135 705  21 769
3 886 990  98 823 586 411 150 783 776 382
4 937 652 540 327 239 741 951  46 681 842
```

As can be seen in the output of the head function all columns of the data frame are named. This is an attribute of data frames. The name of the columns can be accessed and changed with the function `colnames()`. The nameless column to the left is not a real column, it is the name of each row. This can be accessed by the function `rownames()`. The row names is an identifying column, giving a unique ID to each row, similar to the identity column in SQL. However, unlike SQL it does not have to hold a numeric value so long as the values are unique (i.e. rows with duplicate names are not allowed)

```
> colnames(df1)
```

```
[1] "X" "Y" "Z"
```

```
> colnames(df1)[1] <- "Sample1"
```

```
> colnames(df1)
```

```
[1] "Sample1" "Y"      "Z"
```

```
> head(rownames(df2),3)
```

```
[1] "1" "2" "3"
```

```
> rownames(df2) <- LETTERS[1:nrow(df2)]
```

```
> head(rownames(df2),3)
```

```
[1] "A" "B" "C"
```

Data frames behave both like a list and like matrix. Because the columns are

named, they can be accessed by the `$` operator. However, just like matrices they can also be subset by `[]` and operations on data frames can be vectorized.

Let's look at some examples of how to work with data frames.

```
> #accessing a single column using the $ operator
> df1$Sample1
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
> #subsetting a data frame using []
> #If no commas are used this is like subsetting a list
> df1[1:2]
```

	Sample1	Y
1	1	100
2	2	200
3	3	300
4	4	400
5	5	500
6	6	600
7	7	700
8	8	800
9	9	900
10	10	1000

```
> #Like a matrix (i.e. 2D)
> df1[,1]
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
> #First 3 rows, and first 2 columns
> df1[1:3,1:2]
```

	Sample1	Y
1	1	100
2	2	200
3	3	300

```
> #Adding a column to the data frame
> df1$New_column <- df2$V1
>
> #adding a new row to a data frame
> df1[nrow(df1)+1,] <- c(1,1,1,1)
> df1[nrow(df1),]
```

	Sample1	Y	Z	New_column
11	1	1	1	1

```
> #Natural log transformation of a data frame
> df3 <- log(df2)
```

```
> head(df3,3)
```

	V1	V2	V3	V4	V5	V6	V7	V8
A	6.755769	6.107023	4.700480	6.514713	5.446737	6.794587	6.428105	6.881411
B	6.274762	6.431331	6.624065	5.902633	5.056246	6.763885	4.905275	6.558198
C	6.786717	6.897705	4.584967	6.712956	6.373320	6.018593	5.010635	6.663133

	V9	V10
A	5.602119	6.445720
B	3.044522	6.645091
C	6.654153	5.945421

Summary exercise Lab1

Start a script file in RStudio name it 3315_lab1_yourinitials.R You will submit this script and output file if applicable to D2l submission folder. Check your course outline for submission deadline.

1. Create a vector containing the string “How much wood would a woodchuck chuck if a woodchuck could chuck wood? He would chuck, he would, as much as he could, and chuck as much wood As a woodchuck would if a woodchuck could chuck wood”

- How many elements does this vector contain? Show your work.
- Count how many times does the word “Wood” appear in the vector

2. Create 3 identical strings of 10000 randomly selected polynucleotides and assign them to three vectors (do not copy one string three times)

- Show that they are identical
- What is the %GC in the string?

3. Create an empty list of 4 element

4. Name the elements E1-E4

5. Create a matrix from 3 numerical vectors each with 10 elements, assign it to the first element of the list

6. log10 transform the first element of the list and assign it to the 2 element of the list

7. Convert the first element of the list to a dataframe and assign it to the 3 element of the list

8. Assign the dataframe to the 4th element of the list and add a 4th column of 10 letters

- 9.** log10 transform the numeric columns of the dataframe occupying the 4th element of the list
-

Chapter 1

Cross-references

Cross-references make it easier for your readers to find and link to elements in your book.

1.1 Chapters and sub-chapters

There are two steps to cross-reference any heading:

1. Label the heading: `# Hello world {#nice-label}`.
 - Leave the label off if you like the automated heading generated based on your heading title: for example, `# Hello world = # Hello world {#hello-world}`.
 - To label an un-numbered heading, use: `# Hello world {-#nice-label}` or `{# Hello world .unnumbered}`.
2. Next, reference the labeled heading anywhere in the text using `\@ref(nice-label)`; for example, please see Chapter 1.
 - If you prefer text as the link instead of a numbered reference use: any text you want can go here.

1.2 Captioned figures and tables

Figures and tables *with captions* can also be cross-referenced from elsewhere in your book using `\@ref(fig:chunk-label)` and `\@ref(tab:chunk-label)`, respectively.

See Figure 1.1.

```
> par(mar = c(4, 4, .1, .1))
> plot(pressure, type = 'b', pch = 19)
```

Don't miss Table 1.1.

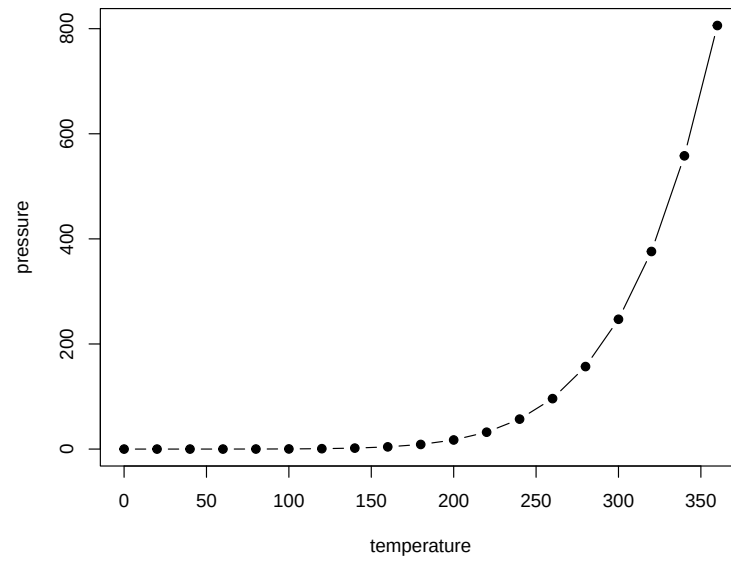


Figure 1.1: Here is a nice figure!

```
> knitr::kable(  
+   head(pressure, 10), caption = 'Here is a nice table!',  
+   booktabs = TRUE  
+ )
```

Table 1.1: Here is a nice table!

temperature	pressure
0	0.0002
20	0.0012
40	0.0060
60	0.0300
80	0.0900
100	0.2700
120	0.7500
140	1.8500
160	4.2000
180	8.8000

Chapter 2

Parts

You can add parts to organize one or more book chapters together. Parts can be inserted at the top of an .Rmd file, before the first-level chapter heading in that same file.

Add a numbered part: `# (PART) Act one {-}` (followed by `# A chapter`)

Add an unnumbered part: `# (PART\*) Act one {-}` (followed by `# A chapter`)

Add an appendix as a special kind of un-numbered part: `# (APPENDIX) Other stuff {-}` (followed by `# A chapter`). Chapters in an appendix are prepended with letters instead of numbers.

Chapter 3

Footnotes and citations

3.1 Footnotes

Footnotes are put inside the square brackets after a caret `^[]`. Like this one ¹.

3.2 Citations

Reference items in your bibliography file(s) using `@key`.

For example, we are using the **bookdown** package [Xie, 2023] (check out the last code chunk in `index.Rmd` to see how this citation key was added) in this sample book, which was built on top of R Markdown and **knitr** [Xie, 2015] (this citation was added manually in an external file `book.bib`). Note that the `.bib` files need to be listed in the `index.Rmd` with the YAML `bibliography` key.

The RStudio Visual Markdown Editor can also make it easier to insert citations: <https://rstudio.github.io/visual-markdown-editing/#/citations>

¹This is a footnote.

Chapter 4

Blocks

4.1 Equations

Here is an equation.

$$f(k) = \binom{n}{k} p^k (1-p)^{n-k} \quad (4.1)$$

You may refer to using `\@ref{eq:binom}`, like see Equation (4.1).

4.2 Theorems and proofs

Labeled theorems can be referenced in text using `\@ref{thm:tri}`, for example, check out this smart theorem 4.1.

Theorem 4.1. *For a right triangle, if c denotes the length of the hypotenuse and a and b denote the lengths of the **other** two sides, we have*

$$a^2 + b^2 = c^2$$

Read more here <https://bookdown.org/yihui/bookdown/markdown-extensions-by-bookdown.html>.

4.3 Callout blocks

The R Markdown Cookbook provides more help on how to use custom blocks to design your own callouts: <https://bookdown.org/yihui/rmarkdown-cookbook/custom-blocks.html>

Chapter 5

Sharing your book

5.1 Publishing

HTML books can be published online, see: <https://bookdown.org/yihui/bookdown/publishing.html>

5.2 404 pages

By default, users will be directed to a 404 page if they try to access a webpage that cannot be found. If you'd like to customize your 404 page instead of using the default, you may add either a `_404.Rmd` or `_404.md` file to your project root and use code and/or Markdown syntax.

5.3 Metadata for sharing

Bookdown HTML books will provide HTML metadata for social sharing on platforms like Twitter, Facebook, and LinkedIn, using information you provide in the `index.Rmd` YAML. To setup, set the `url` for your book and the path to your `cover-image` file. Your book's `title` and `description` are also used.

This `gitbook` uses the same social sharing data across all chapters in your book—all links shared will look the same.

Specify your book's source repository on GitHub using the `edit` key under the configuration options in the `_output.yml` file, which allows users to suggest an edit by linking to a chapter's source file.

Read more about the features of this output format here:

<https://pkgs.rstudio.com/bookdown/reference/gitbook.html>

Or use:

```
> ?bookdown::gitbook
```

Bibliography

Yihui Xie. *Dynamic Documents with R and knitr*. Chapman and Hall/CRC, Boca Raton, Florida, 2nd edition, 2015. URL <http://yihui.org/knitr/>. ISBN 978-1498716963.

Yihui Xie. *bookdown: Authoring Books and Technical Documents with R Markdown*, 2023. URL <https://CRAN.R-project.org/package=bookdown>. R package version 0.32.